

Python Programming Exercises

Program 1: Prime Number Check Using Function

Question: Check whether a given number is prime by defining a function that returns True or False.

Full Code

```
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Line-by-Line Explanation

Code Line	What It Does
def is_prime(n):	Defines a function named is_prime that takes one input, n.
if n <= 1:	Numbers 1 and below are never prime, so this checks for that case first.
return False	If n is 1 or less, the function immediately reports False (not prime).
for i in range(2, int(n**0.5) + 1):	Loops i from 2 up to the square root of n. Only checking up to the square root is enough to prove primality.
if n % i == 0:	Checks if n divides evenly by i (remainder is 0).
return False	If it divides evenly, n has a factor other than 1 and itself, so it is not prime.
return True	If no divisor was found in the loop, n is prime.

Worked Example

Input: is_prime(29)

Trace: 29 <= 1 is False. sqrt(29) ≈ 5.38, so int(5.38)+1 = 6, loop checks i = 2, 3, 4, 5. 29 is not divisible by any of them, so the loop finishes without returning False.

Output: True

Program 2: Fibonacci Numbers Less Than n

Question: Generate all Fibonacci numbers less than a given number n.

Full Code

```
n = int(input())
a, b = 0, 1
while a < n:
    print(a, end=" ")
    a, b = b, a + b
```

Line-by-Line Explanation

Code Line	What It Does
<code>n = int(input())</code>	Reads a value typed by the user and converts it from text to an integer, storing it in <code>n</code> .
<code>a, b = 0, 1</code>	Sets up the first two Fibonacci numbers: <code>a = 0</code> and <code>b = 1</code> .
<code>while a < n:</code>	Keeps repeating the loop as long as the current Fibonacci number <code>a</code> is smaller than <code>n</code> .
<code> print(a, end=" ")</code>	Prints the current value of <code>a</code> , followed by a space instead of a new line.
<code> a, b = b, a + b</code>	Moves forward one step in the sequence: <code>a</code> becomes the old <code>b</code> , and <code>b</code> becomes the sum of the old <code>a</code> and <code>b</code> .

Worked Example

Input: `n = 10`

Trace: *a starts at 0 (printed), then 1, 1, 2, 3, 5, 8. The next value would be 13, but 13 is not less than 10, so the loop stops.*

Output: `0 1 1 2 3 5 8`

Program 3: Second-Largest Element in a List (using `max()`)

Question: Find the second-largest element in a list. Rewritten here to use Python's built-in `max()` function instead of a manual comparison loop.

Full Code

```
lst = list(map(int, input().split()))
largest = max(lst)
second = max(x for x in lst if x != largest)
print(second)
```

Line-by-Line Explanation

Code Line	What It Does
<code>lst = list(map(int, input().split()))</code>	Reads a line of space-separated numbers, converts each piece of text to an integer, and stores them as a list called <code>lst</code> .
<code>largest = max(lst)</code>	Uses the built-in <code>max()</code> function to find the biggest number in the list.
<code>second = max(x for x in lst if x != largest)</code>	Builds a temporary list of every value that is not equal to the largest one, then calls <code>max()</code> again to find the biggest value among those — this is the second-largest.
<code>print(second)</code>	Displays the second-largest value.

Worked Example

Input: `10 20 4 45 99`

Trace: *`max(lst)` finds 99 as the largest. The remaining values are 10, 20, 4, and 45. `max()` on that group gives 45.*

Output: `45`

Program 4: Character Type Count

Question: Count the number of uppercase letters, lowercase letters, digits, and special characters in a given string.

Full Code

```
s = input()
upper = lower = digit = special = 0

for ch in s:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digit += 1
    else:
        special += 1

print(upper, lower, digit, special)
```

Line-by-Line Explanation

Code Line	What It Does
<code>s = input()</code>	Reads a line of text from the user and stores it in <code>s</code> .
<code>upper = lower = digit = special = 0</code>	Creates four counters, all starting at 0, one for each character type.
<code>for ch in s:</code>	Goes through the string one character at a time, calling each one <code>ch</code> .
<code>if ch.isupper():</code>	Checks whether <code>ch</code> is an uppercase letter (A-Z).
<code>upper += 1</code>	If it is, adds 1 to the upper counter.
<code>elif ch.islower():</code>	Otherwise, checks whether <code>ch</code> is a lowercase letter (a-z).
<code>lower += 1</code>	If it is, adds 1 to the lower counter.
<code>elif ch.isdigit():</code>	Otherwise, checks whether <code>ch</code> is a digit (0-9).
<code>digit += 1</code>	If it is, adds 1 to the digit counter.
<code>else:</code>	If none of the above matched, the character must be something else, like punctuation.
<code>special += 1</code>	Adds 1 to the special counter.
<code>print(upper, lower, digit, special)</code>	Prints all four totals, separated by spaces.

Worked Example

Input: "Hello123!"

Trace: *H* is uppercase (1). *e, l, l, o* are lowercase (4). *1, 2, 3* are digits (3). *!* is special (1).

Output: 1 4 3 1

Program 5: Merge Two Lists Without Duplicates

Question: Merge two lists into a single list, removing duplicate elements and preserving the original order.

Full Code

```
l1 = list(map(int, input().split()))
l2 = list(map(int, input().split()))

result = []
for x in l1 + l2:
    if x not in result:
        result.append(x)

print(result)
```

Line-by-Line Explanation

Code Line	What It Does
<code>l1 = list(map(int, input().split()))</code>	Reads the first line of numbers and stores them as a list l1.
<code>l2 = list(map(int, input().split()))</code>	Reads the second line of numbers and stores them as a list l2.
<code>result = []</code>	Creates an empty list that will hold the merged, duplicate-free values.
<code>for x in l1 + l2:</code>	Joins l1 and l2 into one combined list and goes through it one value at a time as x.
<code> if x not in result:</code>	Checks whether x has already been added to result.
<code> result.append(x)</code>	If not, adds x to result — this is what removes duplicates while keeping order.
<code>print(result)</code>	Displays the final merged list.

Worked Example

Input: l1 = 1 2 3, l2 = 3 4 5

Trace: Combined list is [1, 2, 3, 3, 4, 5]. The second 3 is skipped because it's already in result.

Output: [1, 2, 3, 4, 5]

Program 6: Anagram Check

Question: Check whether two strings are anagrams of each other.

Full Code

```
s1 = input().replace(" ", "").lower()
s2 = input().replace(" ", "").lower()

print(sorted(s1) == sorted(s2))
```

Line-by-Line Explanation

Code Line	What It Does
<code>s1 = input().replace(" ", "").lower()</code>	Reads the first string, removes all spaces, and converts it to lowercase so comparisons are fair.
<code>s2 = input().replace(" ", "").lower()</code>	Does the same cleanup for the second string.

Code Line	What It Does
<code>print(sorted(s1) == sorted(s2))</code>	Arranges the letters of each cleaned string alphabetically and checks if the two results are identical — if so, they contain exactly the same letters.

Worked Example

Input: "listen", "silent"

Trace: `sorted("listen")` gives ['e','i','l','n','s','t'], and `sorted("silent")` gives the same letters in the same sorted order.

Output: **True**

Program 7: GCD Using Function

Question: Find the GCD (Greatest Common Divisor) of two numbers using a user-defined function.

Full Code

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print(gcd(12, 18))
```

Line-by-Line Explanation

Code Line	What It Does
<code>def gcd(a, b):</code>	Defines a function called gcd that takes two numbers, a and b.
<code> while b:</code>	Keeps looping as long as b is not zero (this is the Euclidean algorithm).
<code> a, b = b, a % b</code>	Replaces a with b, and replaces b with the remainder of a divided by b.
<code> return a</code>	Once b becomes 0, a holds the greatest common divisor, so it's returned.
<code>print(gcd(12, 18))</code>	Calls the function with 12 and 18, and prints the result.

Worked Example

Input: gcd(12, 18)

Trace: Step 1: $a=12, b=18 \rightarrow a, b = 18, 12\%18=12$. Step 2: $a=18, b=12 \rightarrow a, b = 12, 18\%12=6$. Step 3: $a=12, b=6 \rightarrow a, b = 6, 12\%6=0$. Now $b=0$, loop stops, $a=6$.

Output: **6**

Program 8: Right Rotate a List by k

Question: Rotate a list to the right by k positions, where k is given by the user.

Full Code

```
lst = list(map(int, input().split()))
```

```
k = int(input()) % len(lst)

print(lst[-k:] + lst[:-k])
```

Line-by-Line Explanation

Code Line	What It Does
lst = list(map(int, input().split()))	Reads a line of numbers into a list called lst.
k = int(input()) % len(lst)	Reads how many positions to rotate by. The % len(lst) keeps k within the list's size, so rotating by more than the list length still works correctly.
print(lst[-k:] + lst[:-k])	Takes the last k elements (lst[-k:]) and places them in front, then adds the remaining earlier elements (lst[:-k]) after them.

Worked Example

Input: lst = 1 2 3 4 5, k = 2

Trace: lst[-2:] is [4, 5] and lst[:-2] is [1, 2, 3]. Joining them puts the last two elements first.

Output: [4, 5, 1, 2, 3]

Program 9: Character Frequency Using Dictionary

Question: Find the frequency of each character in a string using a dictionary.

Full Code

```
s = input()
freq = {}

for ch in s:
    freq[ch] = freq.get(ch, 0) + 1

print(freq)
```

Line-by-Line Explanation

Code Line	What It Does
s = input()	Reads a string from the user.
freq = {}	Creates an empty dictionary to store each character and how many times it appears.
for ch in s:	Goes through the string one character at a time.
freq[ch] = freq.get(ch, 0) + 1	Looks up the current count for ch (or 0 if it hasn't appeared yet) and adds 1, storing the updated count back in the dictionary.
print(freq)	Displays the full dictionary of character counts.

Worked Example

Input: "hello"

Trace: h→1, e→1, l→1 then l→2 (seen again), o→1.

Output: {'h': 1, 'e': 1, 'l': 2, 'o': 1}

Program 10: Separate Even and Odd Numbers

Question: Separate even and odd numbers from a list into two different lists.

Full Code

```
lst = list(map(int, input().split()))
even = [x for x in lst if x % 2 == 0]
odd = [x for x in lst if x % 2 != 0]

print(even)
print(odd)
```

Line-by-Line Explanation

Code Line	What It Does
lst = list(map(int, input().split()))	Reads a line of numbers into a list called lst.
even = [x for x in lst if x % 2 == 0]	Builds a new list containing only the values from lst whose remainder when divided by 2 is 0 (even numbers).
odd = [x for x in lst if x % 2 != 0]	Builds another new list containing the values whose remainder is not 0 (odd numbers).
print(even)	Displays the even numbers.
print(odd)	Displays the odd numbers.

Worked Example

Input: 1 2 3 4 5 6

Trace: Even values are 2, 4, 6. Odd values are 1, 3, 5.

Output: [2, 4, 6]
[1, 3, 5]

Program 11: Armstrong Numbers in a Range

Question: Find all Armstrong numbers within a given range.

Full Code

```
for num in range(1, 1000):
    order = len(str(num))
    if num == sum(int(d)**order for d in str(num)):
        print(num)
```

Line-by-Line Explanation

Code Line	What It Does
for num in range(1, 1000):	Checks every whole number from 1 up to 999.
order = len(str(num))	Converts num to text to count how many digits it has (its 'order').

Code Line	What It Does
<code>if num == sum(int(d)**order for d in str(num)):</code>	Raises each digit of num to the power of order, adds those results together, and checks if the total equals num itself.
<code>print(num)</code>	If it matches, num is an Armstrong number, so it's printed.

Worked Example

Input: num = 153

Trace: 153 has 3 digits, so order = 3. $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$, which equals num.

Output: 153 (printed, along with 1, 2, ..., 9, 370, 371, 407)

Program 12: Perfect Number Check

Question: Check whether a number is a perfect number (the sum of its proper divisors equals the number).

Full Code

```
n = int(input())
s = sum(i for i in range(1, n) if n % i == 0)
print(s == n)
```

Line-by-Line Explanation

Code Line	What It Does
<code>n = int(input())</code>	Reads the number to check.
<code>s = sum(i for i in range(1, n) if n % i == 0)</code>	Looks at every number i from 1 up to (but not including) n, keeps the ones that divide n evenly, and adds them all together.
<code>print(s == n)</code>	Compares that sum to n — if they're equal, n is a perfect number.

Worked Example

Input: n = 28

Trace: Divisors of 28 (excluding 28 itself) are 1, 2, 4, 7, 14. Their sum is $1+2+4+7+14 = 28$, which equals n.

Output: True

Program 13: Sort Tuples by Second Element

Question: Sort a list of tuples based on the second element of each tuple.

Full Code

```
lst = [(1, 3), (4, 1), (2, 2)]
lst.sort(key=lambda x: x[1])
print(lst)
```

Line-by-Line Explanation

Code Line	What It Does
<code>lst = [(1, 3), (4, 1), (2, 2)]</code>	Creates a list of tuples, where each tuple has two values.

Code Line	What It Does
<code>lst.sort(key=lambda x: x[1])</code>	Sorts the list in place. The <code>key=lambda x: x[1]</code> tells <code>sort()</code> to compare tuples using their second value (index 1), not the first.
<code>print(lst)</code>	Displays the sorted list.

Worked Example

Input: `[(1, 3), (4, 1), (2, 2)]`

Trace: Comparing second elements: 1 (from (4,1)), 2 (from (2,2)), 3 (from (1,3)) — arranged smallest to largest.

Output: `[(4, 1), (2, 2), (1, 3)]`

Program 14: Decimal to Binary, Octal, and Hexadecimal

Question: Convert a decimal number to binary, octal, and hexadecimal without using built-in conversion functions.

Full Code

```
n = int(input())

def convert(n, base):
    res = ""
    digits = "0123456789ABCDEF"
    while n > 0:
        res = digits[n % base] + res
        n //= base
    return res

print(convert(n, 2), convert(n, 8), convert(n, 16))
```

Line-by-Line Explanation

Code Line	What It Does
<code>n = int(input())</code>	Reads the decimal number to convert.
<code>def convert(n, base):</code>	Defines a function that converts <code>n</code> into any target base (2 for binary, 8 for octal, 16 for hex).
<code> res = ""</code>	Starts an empty string that will build up the converted number.
<code> digits = "0123456789ABCDEF"</code>	A lookup string — the position of each character matches the digit value it represents (useful for hex letters A-F).
<code> while n > 0:</code>	Keeps working as long as there's still a number left to convert.
<code> res = digits[n % base] + res</code>	Finds the remainder of <code>n</code> divided by <code>base</code> , looks up the matching digit character, and places it at the front of <code>res</code> (since digits come out in reverse order).
<code> n //= base</code>	Removes that last digit from <code>n</code> by dividing it down, using integer (whole-number) division.
<code> return res</code>	Once <code>n</code> reaches 0, returns the fully built string.
<code>print(convert(n, 2), convert(n, 8), convert(n, 16))</code>	Calls <code>convert()</code> three times, once per base, and prints all three results together.

Worked Example

Input: n = 29

Trace: Binary: 29 → 11101. Octal: 29 → 35. Hexadecimal: 29 → 1D.

Output: 11101 35 1D

Program 15: First Non-Repeating Character

Question: Find the first non-repeating character in a string.

Full Code

```
s = input()
for ch in s:
    if s.count(ch) == 1:
        print(ch)
        break
```

Line-by-Line Explanation

Code Line	What It Does
s = input()	Reads the string to search.
for ch in s:	Goes through the string one character at a time, in order.
if s.count(ch) == 1:	Counts how many times ch appears anywhere in s. If it appears exactly once, it's non-repeating.
print(ch)	Prints that character.
break	Stops the loop immediately, since we only want the first such character.

Worked Example

Input: "swiss"

Trace: s appears twice, w appears once, i appears once. The first character with a count of 1, scanning left to right, is 'w'.

Output: w

Program 16: Transpose of a Matrix

Question: Compute the transpose of a matrix represented as a list of lists.

Full Code

```
matrix = [[1,2,3],[4,5,6]]
transpose = list(map(list, zip(*matrix)))
print(transpose)
```

Line-by-Line Explanation

Code Line	What It Does
matrix = [[1,2,3],[4,5,6]]	Creates a matrix as a list of rows — here, two rows of three numbers each.

Code Line	What It Does
<code>transpose = list(map(list, zip(*matrix)))</code>	<code>zip(*matrix)</code> unpacks each row and pairs up values column-by-column; <code>map(list, ...)</code> turns each resulting pair into a list instead of a tuple; the outer <code>list(...)</code> collects everything into the final transposed matrix.
<code>print(transpose)</code>	Displays the transposed matrix, where rows and columns have been swapped.

Worked Example

Input: `[[1, 2, 3], [4, 5, 6]]`

Trace: `zip(*matrix)` pairs the first elements of each row together (1,4), then the second elements (2,5), then the third (3,6).

Output: `[[1, 4], [2, 5], [3, 6]]`

Program 17: Password Validation

Question: Validate a password: at least 8 characters, with at least one uppercase letter, one lowercase letter, and one digit.

Full Code

```
pwd = input()

valid = (len(pwd) >= 8 and
        any(c.isupper() for c in pwd) and
        any(c.islower() for c in pwd) and
        any(c.isdigit() for c in pwd))

print(valid)
```

Line-by-Line Explanation

Code Line	What It Does
<code>pwd = input()</code>	Reads the password text to check.
<code>valid = (len(pwd) >= 8 and</code>	Starts building a True/False result. First condition: the password must be at least 8 characters long.
<code>and</code> <code>any(c.isupper() for c in pwd)</code>	Checks that at least one character in <code>pwd</code> is uppercase — <code>any()</code> returns True if even one match is found.
<code>and</code> <code>any(c.islower() for c in pwd)</code>	Checks that at least one character is lowercase.
<code>any(c.isdigit() for c in pwd))</code>	Checks that at least one character is a digit. All four conditions must be True for <code>valid</code> to be True.
<code>print(valid)</code>	Displays whether the password passed every rule.

Worked Example

Input: `"Password1"`

Trace: Length is 9 (`>=8`, pass). Contains 'P' (uppercase, pass). Contains 'assword' (lowercase, pass). Contains '1' (digit, pass). All four checks pass.

Output: `True`

Program 18: Student Marks Dictionary (using max())

Question: Read student names and marks, store them in a dictionary, and display the student with the highest marks.

Full Code

```
students = {"A": 85, "B": 92, "C": 78}
top = max(students, key=students.get)
print(top, students[top])
```

Line-by-Line Explanation

Code Line	What It Does
students = {"A": 85, "B": 92, "C": 78}	Creates a dictionary where each student's name is a key and their marks are the value.
top = max(students, key=students.get)	Uses the built-in max() function on the dictionary's keys (the names). key=students.get tells max() to compare each name by looking up its marks, rather than comparing the names alphabetically.
print(top, students[top])	Prints the top student's name, and then looks up their marks in the dictionary to print alongside it.

Worked Example

Input: {"A": 85, "B": 92, "C": 78}

Trace: max() compares 85, 92, and 78 using students.get as the comparison key, and finds that 'B' has the highest value, 92.

Output: B 92

Program 19: Menu-Driven Calculator Using Functions

Question: Implement a menu-driven calculator using functions for addition, subtraction, multiplication, and division.

Full Code

```
def add(a,b): return a+b
def sub(a,b): return a-b
def mul(a,b): return a*b
def div(a,b): return a/b if b != 0 else None
```

Line-by-Line Explanation

Code Line	What It Does
def add(a,b): return a+b	Defines a function add that takes two numbers and returns their sum.
def sub(a,b): return a-b	Defines a function sub that returns the result of subtracting b from a.
def mul(a,b): return a*b	Defines a function mul that returns the product of a and b.
def div(a,b): return a/b if b != 0 else None	Defines a function div that returns a divided by b, but only if b isn't zero — otherwise it safely returns None instead of crashing with a divide-by-zero error.

Worked Example

Input: `add(4, 5), div(4, 0)`

Trace: *add(4, 5) computes 4 + 5. div(4, 0) checks $b \neq 0$, finds it's False (b is 0), so it returns None instead of dividing.*

Output: 9
None

Program 20: Remove Duplicate Words

Question: Remove all duplicate words from a sentence while maintaining the original word order.

Full Code

```
sentence = input().split()
result = []

for word in sentence:
    if word not in result:
        result.append(word)

print(" ".join(result))
```

Line-by-Line Explanation

Code Line	What It Does
<code>sentence = input().split()</code>	Reads a sentence and breaks it into a list of individual words, splitting on spaces.
<code>result = []</code>	Creates an empty list to collect each word the first time it appears.
<code>for word in sentence:</code>	Goes through the sentence one word at a time, in the original order.
<code>if word not in result:</code>	Checks whether this word has already been added to result.
<code>result.append(word)</code>	If it hasn't, adds it — this keeps only the first occurrence of each word.
<code>print(" ".join(result))</code>	Joins all the kept words back together with single spaces between them and prints the final sentence.

Worked Example

Input: "the cat sat on the mat the cat ran"

Trace: *the cat sat on are all new, so they're kept. The next 'the' is a repeat (skipped), mat is new (kept), the next 'the' and 'cat' are repeats (skipped), ran is new (kept).*

Output: the cat sat on mat ran